

Release Notes

Version 0.4.8:

Bug Fixes

- fix deprecated needs_proba parameter in make_scorer

Version 0.4.7:

Bug Fixes

- fix merge_categorical_columns when there are no cats
- Handle pandas option setting context in case it doesn't exist
- Remove is_categorical_dtype as it is getting deprecated

Improvements

-
-

Version 0.4.6:

Bug Fixes

- should now work with the format of shap 0.45 that returns a three dimensional np.array instead of a list of 2-dimensional np.arrays for classifiers

-

Improvements

- Fixed several pandas warning about to be deprecated behaviours

-

Version 0.4.4:

Breaking Changes

-
-

New Features

-
-

Bug Fixes

- Add warning to set `shap\_kwargs=dict(check\_additivity=True)` for skorch models, and switch this on for the tests.

-

Improvements

-

-

Other Changes

-

-

Version 0.4.3:

Breaking Changes

-

-

New Features

- models that use kernel explainer but output multi-dimensional predictions such as PLSRegression are now supported. Predictions now get squeezed in the kernel function.

-

Bug Fixes

- Fixed bug with pandas v2, Pandas v2 now supported

Improvements

- Fixed a number of user warnings

-

Other Changes

-

-

Version 0.4.2.2:

pins dependencies for flask-wtf>1.1, numpy<1.24 and pandas<2 while working to sort out some compatibility issues.

Version 0.4.2.1:

Bug Fixes

- tries to work around wonky index dropdown search bug introduced by latest dash release.
- Dropdown search now works again, but index propagation is still flaky when number of idxs > max_idx_in_dropdown(1000 by default)
- displays warning to downgrade to dash 2.6.2 when this happens

Improvements

-

-

Other Changes

- applied black to the codebase

-

```

## Version 0.4.2:
### Breaking Changes
- Now needs dtreeviz>2.1, due to the API change with version v2
-

### New Features
-
-

### Bug Fixes
- Fixed import and tree display bug with newer version of dtreeviz
-

### Improvements
-
-

### Other Changes
-
-

### Version 0.4.1

### New Features
- added routes_pathname_prefix:str=None,
requests_pathname_prefix:str=None, to ExplainerDashboard to help running
the dashboard on e.g. Sagemaker
-

### Bug Fixes
- Bug with plotly `showticklabels=False` changed to
`tickfont=dict(color="rgba(0, 0, 0, 0)")`
- Imports now comply with dtreeviz v2 API


## Version 0.4.0: upgrade bootstrap5, drop python 3.6 and 3.7 support and
improved pipeline support
- Upgrades the dashboard to `bootstrap5` and `dash-bootstrap-components`
`v1` (which is also based on bootstrap5), this
    may break older custom dashboards that included bootstrap5 components
from `dash-bootstrap-components<1`
- Support terminated for python `3.6` and `3.7` as the latest version of
`scikit-learn` (1.1) dropped support as well
    and explainerdashboard depends on the improved pipeline feature
naming in `scikit-learn>=1.1`

### New Features
- Better support for large datasets through dynamic server-side index
dropdown option selection. This means that not all indexes have to be
stored client side in the browser, but

```

get rather automatically updated as you start typing. This should help especially with large datasets with large number of indexes.

This new server-side dynamic index dropdowns get activated if the number of rows > `max\_idx\_in\_dropdown` (defaults to 1000).

- Both sklearn and imblearn Pipelines are now supported with automated feature names generated, as long as all the transformers have a `.get\_feature\_names\_out()` method
- Adds `shap\_kwargs` parameter to the explainers that allow you to pass additional kwargs to the shap values generating call, e.g. `shap\_kwargs=dict(check\_additivity=False)`
- Can now specify absolute path with `explainerfile\_absolute\_path` when dumping `dashboard.yaml` with `db.to\_yaml(...)`

Bug Fixes

- Suppresses warnings when extracting final model from pipeline that was not fitted on a dataframe.

-

Improvements

- No longer limiting werkzeug version due to upstream bug fixes of `dash` and `jupyter-dash`

-

Other Changes

- Some dropdowns now better aligned.

-

Version 0.3.8.1:

Breaking Changes

-

-

New Features

- Adds support for sklearn Pipelines that add new features (such as those including OneHotEncoder)

as long as they support the new `get\_features\_out()` method. Not all estimators and transformers

have this method implemented yet, but if all estimators in your pipeline do, then

explainerdashboard will extract the final dataframe and the model from your pipelines.

For now this does result in a lot of "this model was fitted on a numpy array but you provided a dataframe"

warnings.

-

Bug Fixes

- Fixed a bug with sorting pdp features
- Fixed werkzeug<=2.0.3 due to some new features that broke JupyterDash
- Changes use of pd.append that will be deprecated soon and is currently generated warnings.

Improvements

-

```

-

### Other Changes
-
-

## Version 0.3.8:
### Breaking Changes
- Forces dash v2 dependency
### Bug Fixes
- fixes bug introduced by breaking change in pandas 1.40
### Other Changes
- Switches to dash v2 style imports

## Version 0.3.7
### Breaking Changes
-
-

### New Features
- Export your ExplainerHub to static html with `hub.to_html()` and
`hub.save_html()` methods
- Export your ExplainerHub to a zip file with static html exports with
`to_zip()` method
- Manually add pre-calculated shap values with
`explainer.set_shap_values()`
- Manually add pre-calculated shap interaction values with
`explainer.set_shap_interaction_values()`

### Bug Fixes
- Fixed bug with What if tab components static html export (missing
`</div>`)
-

### Improvements
-
-

### Other Changes
-
-

## Version 0.3.6:

### New Features
- Static html export! You can export a static version of the dashboard
using the default values
    that you specified in the components or through kwargs with
`dashboard.to_html()`.
    - for custom components you need to define your own custom
`to_html()` methods, see the documentation.
- A toggle is added to the dashboard header that allows you to download a
static export

```

- of the current live state of the dashboard.
- adds a new toggle and parameter to the ConfusionmatrixComponent to either average
 - the percentage over the entire matrix, over the rows or over the columns.
- Set normalize='all', normalize='true', or normalize='pred'.
- also adds a `save\_html(filename)` method to all `ExplainerComponents` and `ExplainerDashboard`
- `ExplainerHub` adds a new parameter `index\_to\_base\_route`:
 - Dispatches Hub to `/base\_route/index` instead of the default `/` and `/index`.
 - Useful when the host root is not reserved for the ExplainerHub

Version 0.3.5:

Breaking Changes

-
-

New Features

- adds support for `PyTorch` Neural Networks! (as long as they are wrapped by `skorch`)
- adds `SimplifiedClassifierComposite` and `SimplifiedRegressionComposite` to `explainerdashboard.custom`
- adds flag `simple=True` to load these simplified one page dashboards: `ExplainerDashboard(explainer, simple=True)`
- adds support for visualizing trees of `ExtraTreesClassifier` and `ExtraTreesRegressor`
- adds `FeatureDescriptionsComponent` to `explainerdashboard.custom` and the Importances tab
- adds possibility to dynamically add new dashboards to running ExplainerHub using `/add\_dashboard` route
 - with `add\_dashboard\_route=True` (will only work if you're running the Hub as a single worker/node though!)

Bug Fixes

-
-

Improvements

- `ExplainerDashboard.to\_yaml("dashboards/dashboard.yaml", dump\_explainer=True)`
 - will now dump the explainer in the correct subdirectory (and also default to explainer.joblib)

-

Other Changes

-
-

Version 0.3.4:

```

### Bug Fixes
- Fixes incompatibility bug with dtreeviz >= 1.3
-

### Improvements
- raises ValueError when passing `shap='deep'` as it is not yet correctly
supported
-

## Version 0.3.3:

Highlights:
* Adding support for cross validated metrics
* Better support for pipelines by using kernel explainer
* Making explainer threadsafe by adding locks
* Remove outliers from shap dependence plots

### Breaking Changes
- parameter `permutation_cv` has been deprecated and replaced by
parameter `cv` which
    now also works to calculate cross-validated metrics besides cross-
validated
    permutation importances.

### New Features
- metrics now get calculated with cross validation over `X` when you pass
the
    `cv` parameter to the explainer, this is useful when for some reason
you
    want to pass the training set to the explainer.
- adds winsorization to shap dependence and shap interaction plots
- If `shap='guess'` fails (unable to guess the right type of shap
explainer),
    then default to the model agnostic `shap='kernel'`.
- Better support for sklearn `Pipelines`: if not able to extract
transformer+model,
    then default to `shap.KernelExplainer` to explain the entire pipeline
- you can now remove outliers from shap dependence/interaction plots with
    `remove_outliers=True`: filters all outliers beyond 1.5*IQR

### Bug Fixes
- Sets proper `threading.Lock` before making calls to shap explainer
to prevent race
    conditions with dashboards calling for shap values in multiple
threads.
    (shap is unfortunately not threadsafe)
-

### Improvements
- single shap row KernelExplainer calculations now go without tqdm
progress bar
- added cutoff tpr and fpr to roc auc plot
- added cutoff precision and recall to pr auc plot
- put a loading spinner on shap contrib table

```

Other Changes

-
-

Version 0.3.2.2:

`index\_dropdown=False` now works for indexes not listed in
`set\_index\_list\_func()`

as long as it can be found by `set\_index\_exists\_func`

New Features

- adds `set\_index\_exists\_func` to add function that checks for index
existing

besides those listed by `set\_index\_list\_func()`

Bug Fixes

- bug fix to make `shap.KernelExplainer` (used with explainer
parameter `shap='kernel'`)

work with `RegressionExplainer`

- bug fix when no explicit `labels` are based with index selector

- component only update if `explainer.index\_exists()`: no

`IndexNotFoundError` anymore.

- fixed title for regression index selector labeled 'Custom' bug

- `get\_y()` now returns `.item()` when necessary

- removed ticks from confusion matrix plot when no `labels` param passed

(this bug got reintroduced in recent plotly release)

Improvements

- new helper function `get\_shap\_row(index)` to calculate or look up a
single

row of shap values.

Version 0.3.2:

Highlights:

- Control what metrics to show or use your own custom metrics using
`show\_metrics`

- Set the naming for onehot features with all `0`'s with `cats\_notencoded`

- Speed up plots by displaying only a random sample of markers in scatter
plots with `plot\_sample`.

- make index selection a free text field with `index\_dropdown=False`

New Features

- new parameter `show\_metrics` for both `explainer.metrics()`,
`ClassifierModelSummaryComponent`

and `RegressionModelSummaryComponent`:

- pass a list of metrics and only display those metrics in that order

- you can also pass custom scoring functions as long as they

are of the form `metric\_func(y\_true, y\_pred)`:

`show\_metrics=[metric\_func]`

- For `ClassifierExplainer` what is passed to the custom metric
function

depends on whether the function takes additional parameters

`cutoff`

```

        and `pos_label`. If these are not arguments, then
`y_true=self.y_binary(pos_label)`
        and `y_pred=np.where(self.pred_probas(pos_label)>cutoff, 1,
0)``.
        Else the raw `self.y` and `self.pred_probas` are passed for
the
        custom metric function to do something with.
- custom functions are also stored to `dashboard.yaml` and
imported upon
        loading `ExplainerDashboard.from_config()``
- new parameter `cats_notencoded`: a dict to indicate how to name the
value
        of a onehotencoded features when all onehot columns equal 0. Defaults
to `'NOT_ENCODED'`, but can be adjusted with this parameter. E.g.
`cats_notencoded=dict(Deck="Deck not known")`.
- new parameter `plot_sample` to only plot a random sample in the various
scatter plots. When you have a large dataset, this may significantly
speed up various plots without sacrificing much in expressiveness:
`ExplainerDashboard(explainer, plot_sample=1000).run`
- new parameter `index_dropdown=False` will replace the index dropdowns
with a
        free text field. This can be useful when you have a lot of potential
indexes,
        and the user is expected to know the index string.
        Input will be checked for validity with
`explainer.index_exists(index)`,
        and field indicates when input index does not exist. If index does
not exist,
        will not be forwarded to other components, unless you also set
`index_check=False`.
- adds mean absolute percentage error to the regression metrics. If it is
too
        large a warning will be printed. Can be excluded with the new
`show_metrics`
        parameter.

### Bug Fixes
- `get_classification_df` added to `ClassificationComponent`
dependencies.
-

### Improvements
- accepting single column `pd.DataFrame` for `y`, and automatically
converting
        it to a `pd.Series`
- if WhatIf `FeatureInputComponent` detects the presence of missing
onehot features
        (i.e. rows where all columns of the onehotencoded feature equal 0),
then
        adds `'NOT_ENCODED'` or the matching value from `cats_notencoded` to
the
        dropdown options.
- Generating `name` for parameters for `ExplainerComponents` for which no

```

name is given is now done with a determinative process instead of a random

`uuid`. This should help with scaling custom dashboards across cluster

deployments. Also drops `shortuuid` dependency.

- `ExplainerDashboard` now prints out local ip address when starting dashboard.

- `get\_index\_list()` is only called once upon starting dashboard.

Other Changes

-
-

Version 0.3.1:

This version is mostly about pre-calculating and optimizing the classifier statistics

components. Those components should now be much more responsive with large datasets.

New Features

- new methods `roc\_auc\_curve(pos\_label)` and `pr\_auc\_curve(pos\_label)`

- new method `get\_classification\_df(...)` to get dataframe with number of labels

above and below a given cutoff.

- this now gets used by `plot\_classification(...)`

- new method `confusion\_matrix(cutoff, binary, pos\_label)`

- added parameters `sort\_features` to `FeatureInputComponent`:

- defaults to `'shap'`: order features by mean absolute shap

- if set to `'alphabet'` features are sorted alphabetically

- added parameter `fill\_row\_first` to `FeatureInputComponent`:

- defaults to `True`: fill first row first, then next row, etc

- if False: fill first column first, then second column, etc

Bug Fixes

- categorical mappings now updateable with pandas<=1.2 and python==3.6

- title now overridable for `RegressionRandomIndexComponent`

- added assert check on `summary\_type` for `ShapSummaryComponent`

Improvements

- pre-Calculating lift_curve_df only once and then storing for each pos_label

- plus: storing only 100 evenly spaced rows of lift_curve_df

- dashboard should be more responsive for large datasets

- pre-calculating roc_auc_curve and pr_auc_curve

- dashboard should be more responsive for large datasets

- pre-calculating confusion matrices

- dashboard should be more responsive for large datasets

- pre-calculating classification_dfs

- dashboard should be more responsive for large datasets

- confusion matrix: added axis title, moved predicted labels to bottom of graph

- precision plot: when only adjusting cutoff, simply updating the cutoff line, without recalculating the plot.

Other Changes

-
-

version 0.3.0.1:

Breaking Changes

- new dependency requirements `pandas>=1.2` also implies `python>=3.7`

Bug Fixes

- updates `pandas` version to be compatible with categorical feature operations
- updates dtreeviz version to make `xgboost` and `pyspark` dependencies optional

Improvements

-
-

Other Changes

-
-

version 0.3.0:

This is a major release and comes with lots of breaking changes to the lower level

`ClassifierExplainer` and `RegressionExplainer` API. The higherlevel `ExplainerComponent` and `ExplainerDashboard` API has not been changed however, except for the deprecation of the `cats` and `hide\_cats` parameters.

Explainers generated with version `explainerdashboard <= 0.2.20.1` will not work

with this version, so if you have stored explainers to disk you either have to

rebuild them with this new version, or downgrade back to

`explainerdashboard==0.2.20.1`!

(hope you pinned your dependencies in production! ;)

Main motivation for these breaking changes was to improve memory usage of the

dashboards, especially in production. This lead to the deprecation of the dual cats grouped/not grouped functionality of the dashboard. Once I had committed

to that breaking change, I decided to clean up the entire API and do all the

needed breaking changes at once.

Breaking Changes

- onehot encoded features are now merged by default. This means that the `cats=True`

parameter has been removed from all explainer methods, and the `group cats`

toggle has been removed from all `ExplainerComponents`. This saves both on code complexity and memory usage. If you wish to see the individual contributions of onehot encoded columns, simply don't pass them to the

the `cats` parameter upon construction.

- Deprecated explainer attributes:

- `BaseExplainer`:
 - `self.shap\_values\_cats`
 - `self.shap\_interaction\_values\_cats`
 - `permutation\_importances\_cats`
 - `self.get\_dfs()`
 - `formatted\_contrib\_df()`
 - `self.to\_sql()`
 - `self.check\_cats()`
 - `equivalent\_col`
- `ClassifierExplainer`:
 - `get\_prop\_for\_label`

- Naming changes to attributes:

- `BaseExplainer`:
 - `importances\_df()` -> `get\_importances\_df()`
 - `feature\_permutations\_df()` -> `get\_feature\_permutations\_df()`
 - `get\_int\_idx(index)` -> `get\_idx(index)`
 - `importances\_df()` -> `get\_importances\_df()`
 - `contrib\_df()` -> `get\_contrib\_df()` *
 - `contrib\_summary\_df()` -> `self.get\_summary\_contrib\_df()` *
 - `interaction\_df()` -> `get\_interactions\_df()` *
 - `shap\_values` -> `get\_shap\_values\_df`
 - `plot\_shap\_contributions()` -> `plot\_contributions()`
 - `plot\_shap\_summary()` -> `plot\_importances\_detailed()`
 - `plot\_shap\_dependence()` -> `plot\_dependence()`
 - `plot\_shap\_interaction()` -> `plot\_interaction()`
 - `plot\_shap\_interaction\_summary()` -> `plot\_interactions\_detailed()`
 - `plot\_interactions()` -> `plot\_interactions\_importance()`
 - `n\_features()` -> `n\_features`
 - `shap\_top\_interaction()` -> `top\_shap\_interactions`
 - `shap\_interaction\_values\_by\_col()` -> `shap\_interactions\_values\_for\_col()`
- `ClassifierExplainer`:
 - `self.pred\_proba` -> `self.pred\_proba`
 - `precision\_df()` -> `get\_precision\_df()` *
 - `lift\_curve\_df()` -> `get\_liftcurve\_df()` *
- `RandomForestExplainer`/`XGBExplainer`:
 - `decision\_trees` -> `shadow\_trees`
 - `decisiontree\_df()` -> `get\_decisionpath\_df()`
 - `decisiontree\_summary\_df()` -> `get\_decisionpath\_summary\_df()`
 - `decision\_path\_file()` -> `decisiontree\_file()`
 - `decision\_path()` -> `decisiontree()`
 - `decision\_path\_encoded()` -> `decisiontree\_encoded()`

New Features

- new `Explainer` parameter `precision`: defaults to `'float64'`. Can be set to
 `'float32'` to save on memory usage: `ClassifierExplainer(model, X, y, precision='float32')`
- new `memory\_usage()` method to show which internal attributes take the most memory.
- for multiclass classifiers: `keep\_shap\_pos\_label\_only(pos\_label)` method:
 - drops shap values and shap interactions for all labels except `pos\_label`
 - this should significantly reduce memory usage for multi class classification models.
 - not needed for binary classifiers.
- added `get\_index\_list()`, `get\_X\_row(index)`, and `get\_y(index)` methods.
 - these can be overridden with `.set\_index\_list\_func()`,
 `.set\_X\_row\_func()`
 and `.set\_y\_func()`.
 - by overriding these functions you can for example sample observations from a database or other external storage instead of from
 `X\_test`, `y\_test`.
- added `Popout` buttons to all the major graphs that open a large modal showing just the graph. This makes it easier to focus on a particular graph without distraction from the rest of the dashboard and all it's toggles.
- added `max\_cat\_colors` parameters to `plot\_importance\_detailed` and `plot\_dependence` and `plot\_interactions\_detailed`
 - prevents plotting getting slow with categorical features with many categories.
 - defaults to `5`
 - can be set as `\*\*kwargs` to `ExplainerDashboard`
- adds category limits and sorting to `RegressionVsCol` component
- adds property `X\_merged` that gives a dataframe with the onehot columns merged.

Bug Fixes

- shap dependence: when no point cloud, do not highlight!
- Fixed bug with calculating contributions plot/table for whatif component,
 when InputFeatures had not fully loaded, resulting in shap error.

Improvements

- saving `X.copy()`, instead of using a reference to `X`
 - this would result in more memory usage in development though, so you can `del X\_test` to save memory.
- `ClassifierExplainer` only stores shap (interaction) values for the positive
 - class: shap values for the negative class are generated on the fly by multiplying with `-1`.
- encoding onehot columns as `np.int8` saving memory usage
- encoding categorical features as `pd.category` saving memory usage

- added base ``TreeExplainer`` class that ``RandomForestExplainer`` and ``XGBExplainer`` both derive from
 - will make it easier to extend tree explainers to other models in the future
 - e.g. catboost and lightgbm
- got rid of the callable properties (that were there to assure backward compatibility),
 - and replaced them with regular methods.

Other Changes

-
-

0.2.20.1:

Bug Fixes

- fixes bug allowing single list of logins for `ExplainerDashboard` when passed
 - on to `ExplainerHub`
- fixes bug with explainer generated with `explainerdashboard < version 0.2.20`
 - that did not have a `onehot_cols` property

0.2.20:

Breaking Changes

- ``WhatIfComponent`` deprecated. Use ``WhatIfComposite`` or connect yourself to a ``FeatureInputComponent``
- renaming properties:
 - ``explainer.cats`` -> ``explainer.onehot_cols``
 - ``explainer.cats_dict`` -> ``explainer.onehot_dict``

New Features

- Adds support for model with categorical features that were not onehot encoded
 - (e.g. CatBoost)
- Adds filter on number of categories to display in violin plots and pdp plot,
 - and how to sort the categories (alphabetical, by frequency or by mean abs shap)

Bug Fixes

- fixes bug where str tab indicators returned e.g. the old `ImportancesTab` instead of `ImportancesComposite`
-

Improvements

- No longer depending on PDPbox dependency: built own partial dependence
 - functions with categorical feature support
- autodetect `xgboost.core.Booster` or `lightgbm.Booster` and give `ValueError` to

use the sklearn compatible wrappers instead.

Other Changes

- Introduces list of categorical columns: ``explainer.categorical_cols``
- Introduces dictionary with categorical columns categories: ``explainer.categorical_dict``
- Introduces list of all categorical features: ``explainer.cat_cols``

0.2.19

Breaking Changes

- ExplainerHub: parameter ``user_json`` is now called ``users_file`` (and default to a ``users.yaml`` file)
- Renamed a bunch of ``ExplainerHub`` private methods:
 - ``_validate_user_json`` -> ``_validate_users_file``
 - ``_add_user_to_json`` -> ``_add_user_to_file``
 - ``_add_user_to_dashboard_json`` -> ``_add_user_to_dashboard_file``
 - ``_delete_user_from_json`` -> ``_delete_user_from_file``
 - ``_delete_user_from_dashboard_json`` -> ``_delete_user_from_dashboard_file``

New Features

- Added NavBar to ``ExplainerHub``
- Made ``users.yaml`` to default file for storing users and hashed passwords
 - for ``ExplainerHub`` for easier manual editing.
- Added option ``min_height`` to ``ExplainerHub`` to set the size of the `iFrame`` containing the dashboard.
- Added option ``fluid=True`` to ``ExplainerHub`` to stretch bootstrap container to width of the browser.
- added parameter ``bootstrap`` to ``ExplainerHub`` to override default bootstrap theme.
- added option ``dbs_open_by_default=True`` to ``ExplainerHub`` so that no login is required for dashboards for which there wasn't a specific lists of users declared through ``db_users``. So only dashboards for which users have been defined are password protected.
- Added option ``no_index`` to ``ExplainerHub`` so that no flask route is created for index ``"/``, so that you can add your own custom index. The dashboards are still loaded on their respective routes, so you can link to them or embed them in iframes, etc.
- Added a "wizard" perfect prediction to the lift curve.
 - hide with ``hide_wizard=True`` default to not show with ``wizard=False``.

Bug Fixes

- ``ExplainerHub.from_config()`` now works with non-cwd paths
- ``ExplainerHub.to_yaml("subdirectory/hub.yaml")`` now correctly stores the `users.yaml` file in the correct subdirectory when specified.

```

### Improvements
- added a "powered by: explainerdashboard" footer. Hide it with
hide_poweredby=True.
- added option "None" to shap dependence color col. Also removes the
point cloud
    from the violin plots for categorical features.
- added option `mode` to `ExplainerDashboard.run()` that can override
`self.mode`.

### Other Changes
-

## 0.2.18.1:

### Breaking Changes
-
-

### New Features
- `ExplainerHub` now does user management through `Flask-Login` and a
`user.json` file
- adds an `explainerhub` cli to start explainerhubs and do user
management.

### Bug Fixes
-
-

### Improvements
-
-

### Other Changes
-
-

## 0.2.17:
### Breaking Changes
-
-

### New Features
- Introducing `ExplainerHub`: combine multiple dashboards together behind
a single frontend with convenient url paths.
    - example:
      ```python
 db1 = ExplainerDashboard(explainer, title="Dashboard One",
name='dashboard1')
 db2 = ExplainerDashboard(explainer, title="Dashboard Two",
name='dashboard2')

 hub = ExplainerHub([db1, db2])

```

```

hub.run()

store an recover from config:
hub.to_yaml("hub.yaml")
hub2 = ExplainerHub.from_config("hub.yaml")
```

- adds option `dump_explainer` to `ExplainerDashboard.to_yaml` to automatically dump the explainerfile along with the yaml.
- adds option `use_waitress` to `ExplainerDashboard.run()` and `ExplainerHub.run()`, to use the `waitress` python webserver instead of the `Flask` development server
- adds parameters to `ExplainerDashboard`:
  - `name`: this will be used to assign a url for `ExplainerHub`
  - `description`: this will be used for the title tooltip in the dashboard and in the `ExplainerHub` frontend.

### Bug Fixes
-
-

### Improvements
- the `cli` now uses the `waitress` server by default.
-

### Other Changes
-
-

## Version 0.2.16.2:

### Bug fix/Improvement
- Makes component `name` property for the default composites deterministic instead of random uuid, now also working when loading a dashboard
  .from_config()
  - note however that for custom `ExplainerComponents` the user is still responsible for making sure that all subcomponents get assigned a deterministic `name` (otherwise random uuid names get assigned at dashboard start, which might differ across nodes in e.g. docker swarm deployments)
- Calling `self.register_components()` no longer necessary.

## Version 0.2.16.1:

### Bug fix/Improvement
- Makes component `name` property for the default composites deterministic instead of random uuid.
  This should help remedy bugs with deployment using e.g. docker swarm.

```

- When you pass a list of `ExplainerComponents` to ExplainerDashboard the tabs will get names `1`, `2`, `3`, etc.
- If you then make sure that subcomponents get passed a name like `name=self.name+"1"`, then subcomponents will have deterministic names as well.
- this has been implemented for the default `Composites` that make up the default `explainerdashboard`

Version 0.2.16:

Breaking Changes

- `hide\_whatifcontribution` parameter now called `hide\_whatifcontributiongraph`

-

New Features

- added parameter `n\_input\_cols` to FeatureInputComponent to select in how many columns to split the inputs
- Made PredictionSummaryComponent and ShapContributionTableComponent also work with InputFeatureComponent
- added a PredictionSummaryComponent and ShapContributionTableComponent to the "what if" tab

Bug Fixes

-

-

Improvements

- features of `FeatureInputComponent` are now ordered by mean shap importance
- Added range indicator for numerical features in FeatureInputComponent
 - hide them `hide\_range=True`
- changed a number of dropdowns from `dcc.Dropdown` to `dbc.Select`
- reordered the regression random index selector component a bit

Other Changes

-

-

Version 0.2.15:

Breaking Changes

-

-

New Features

- can now hide entire components on tabs/composites:

...

```
ExplainerDashboard(explainer,
    # importances tab:
    hide_importances=True,
    # classification stats tab:
    hide_globalcutoff=True, hide_modelsummary=True,
```

```

hide_confusionmatrix=True, hide_precision=True,
hide_classification=True, hide_rocauc=True,
hide_prauc=True, hide_liftcurve=True, hide_cumprecision=True,
# regression stats tab:
# hide_modelsummary=True,
hide_predsvsactual=True, hide_residuals=True,
hide_regvscol=True,
# individual predictions:
hide_predindexselector=True, hide_predictionsummary=True,
hide_contributiongraph=True, hide_pdp=True,
hide_contributiontable=True,
# whatif:
hide_whatifindexselector=True, hide_inputeditor=True,
hide_whatifcontribution=True, hide_whatifpdp=True,
# shap dependence:
hide_shapsummary=True, hide_shapdependence=True,
# shap interactions:
hide_interactionssummary=True, hide_interactiondependence=True,
# decisiontrees:
hide_treeindexselector=True, hide_treesgraph=True,
hide_treepathtable=True, hide_treepathgraph=True,
).run()
...
-

### Bug Fixes
- Fixed bug where if you passed a default index as **kwarg, the random
index selector
    would still fire at startup, overriding the passed index
- Fixed bug where in case of ties in shap values the contributions
graph/table would show
    more than depth/topx feature
- Fixed bug where favicon was not showing when using custom bootstrap
theme
- Fixed bug where logodds were multiplied by 100 in
ShapContributionTableComponent

### Improvements
- added checks on `logins` parameter to give more helpful error messages
    - also now accepts a single pair of logins: `logins=['user1',
'password1']`
- added a `hide_footer` parameter to components with a CardFooter

### Other Changes
-
-

## Version 0.2.14:
### Breaking Changes
-
-

### New Features

```

- added `bootstrap` parameter to dashboard to make theming easier:
e.g. `ExplainerDashboard(explainer, bootstrap=dbc.themes.FLATLY).run()`
- added `hide\_subtitle=False` parameter to all components with subtitles
- added `description` parameter to all components to adjust the hover-over-title tooltip
- can pass additional *kwargs to ExplainerDashboard.from_config() to override stored parameters, e.g. `db = ExplainerDashboard.from\_config("dashboard.yaml", higher\_is\_better=False)`

Bug Fixes

- fixed bug where `drop\_na=True` for `explainer.plot\_pdp()` was not working.
-

Improvements

- `\*\*kwargs` are now also stored when calling ExplainerDashboard.to_yaml()
- turned single radioitems into switches
- RegressionVsColComponent: hide "show point cloud next to violin" switch when feature is not in `cats`

Other Changes

-

Version 0.2.13.2

Bug Fixes

- fixed RegressionRandomIndexComponent bug that crashed when `y.astype(np.int64)`, now casting all slider ranges to float.

Version 0.2.13.1

Bug Fixes

- fixed pdp bug introduced with setting `X.index` to `self.idx` where the highlighted index was not the right index
- now hiding entire `CardHeader` when `hide\_title=True`
- index was not initialized in ShapContributionsGraphComponent and ShapContributionsTableComponent

Version 0.2.13:

Breaking Changes

- Now always have to pass a specific port when terminating a JupyterDash-based (i.e. inline, external or jupyterlab) dashboard:
ExplainerDashboard.terminate(port=8050)
- but now also works as a classmethod, so don't have to instantiate an

```

        actual dashboard just to terminate one!
- ExplainerComponent `_register_components` has been renamed to
`component_callbacks`
  to avoid the confusing underscore

### New Features
- new:
`ClassifierPredictionSummaryComponent`, `RegressionPredictionSummaryComponent`
  - already integrated into the individual predictions tab
  - also added a piechart with predictions
- Wrapped all the ExplainerComponents in `dbc.Card` for a cleaner look to
the dashboard.
- added subtitles to all components

### Bug Fixes
-
-

### Improvements
- using `go.Scattergl` instead of `go.Scatter` for some plots which
should improve
  performance with larger datasets
- `ExplainerDashboard.terminate()` is now a classmethod, so don't have to
build
  an ExplainerDashboard instance in order to terminate a running
JupyterDash
  dashboard.
- added `disable_permutations` boolean argument to `ImportancesComponent`
(that
  you can also pass to `ExplainerDashboard` `**kwargs`)
-

### Other Changes
- Added warning that kwargs get passed down the ExplainerComponents
- Added exception when trying to use `ClassifierRandomIndexComponent`
with a
  `RegressionExplainer` or `RegressionRandomIndexComponent` with a
`ClassifierExplainer`
- dashboard now uses Composites directly instead of the ExplainerTabs

## Version 0.2.12:
### Breaking Changes
- removed `metrics_markdown()` method. Added `metrics_descriptions()`
that
  describes the metric in words.
- removed `PredsVsColComponent`, `ResidualsVsColComponent` and
`ActualVsColComponent`,
  these three are now subsumed in `RegressionVsColComponent`.

### New Features
- Added tooltips everywhere throughout the dashboard to explainer

```

components, plots, dropdowns and toggles of the dashboard itself.

Bug Fixes

-
-

Improvements

- changed colors on contributions graph up=green, down=red
 - added `higher\_is\_better` parameter to switch green and red colors.
- Clarified wording on index selector components
- hiding `group cats` toggle everywhere when no cats are passed
- passing `\*\*kwargs` of ExplainerDashboard down to all all tabs and (sub) components
 - so that you can configure components from an ExplainerDashboard param.
 - e.g. `ExplainerDashboard(explainer, higher\_is\_better=False).run()` will pass the higher_is_better param down to all components. In the case of the ShapContributionsGraphComponent and the XGBoostDecisionTrees component this will cause the red and green colors to flip (normally green is up and red is down.)

Other Changes

-
-

Version 0.2.11:

Breaking Changes

-
-

New Features

- added (very limited) sklearn.Pipeline support. You can pass a Pipeline as `model` parameter as long as the pipeline either:
 1. Does not add, remove or reorders any input columns
 2. has a .get_feature_names() method that returns the new column names(this is currently being debated in sklearn SLEP007)
- added cutoff slider to CumulativePrecisionComponent
- For RegressionExplainer added ActualVsColComponent and PredsVsColComponent
 - in order to investigate partial correlations between y/preds and various features.
- added `index\_name` parameter: name of the index column (defaults to `X.index.name` or `idxs.name`). So when you pass `index\_name="Passenger"`, you get a "Random Passenger" button on the index selector instead of "Random Index", etc.

Bug Fixes

- Fixed a number of bugs for when no labels are passed (`y=None`):
 - fixing explainer.random_index() for when y is missing
 - Hiding label/y/residuals selector in RandomIndexSelectors
 - Hiding y/residuals in prediction summary
 - Hiding model_summary tab
 - Removing permutation importances from dashboard

Improvements

- Separated labels for "observed" and "average prediction" better in tree plot
- Renamed "actual" to "observed" in prediction summary
- added unique column check for whatif-component with clearer error message
- model metrics now formatted in a nice table
- removed most of the loading spinners as most graphs are not long loads anyway.

Other Changes

-
-

Version 0.2.10:

New Features

- Explainer parameter `cats` now takes dicts as well where you can specify your own groups of onehotencoded columns.
 - e.g. instead of passing `cats=['Sex']` to group `['Sex\_female', 'Sex\_male', 'Sex\_nan']` you can now do this explicitly: `cats={'Gender':['Sex\_female', 'Sex\_male', 'Sex\_nan']}`
 - Or combine the two methods:
`cats=[{'Gender':['Sex\_female', 'Sex\_male', 'Sex\_nan']}, 'Deck', 'Embarked']`

Version 0.2.9:

Breaking Changes

New Features

- You don't have to pass the list of subcomponents in `self.register\_components()` anymore: it will infer them automatically from `self.\_\_dict\_\_`.

Improvements

- ExplainerComponents now automatically stores all parameters to attributes
- ExplainerComponents now automatically stores all parameters to a ._stored_params dict
- ExplainerDashboard.to_yaml() now support instantiated tabs and stores parameters to yaml

- ExplainerDashboard.to_yaml() now stores the import requirements of subcomponents
- ExplainerDashboard.from_config() now instantiates tabs with stored parameters
- ExplainerDashboard.from_config() now imports classes of subcomponents

Other Changes

- added docstrings to explainer_plots
- added screenshots of ExplainerComponents to docs
- added more gifs to the documentation

Version 0.2.8:

Breaking Changes

- split explainerdashboard.yaml into a explainer.yaml and dashboard.yaml
- Changed UI of the explainerdashboard CLI to reflect this
- This will make it easier in the future to have automatic rebuilds and redeploys
when an modelfile, datafile or configuration file changes.

New Features

- Load an ExplainerDashboard from a configuration file with the classmethod,
e.g. : `ExplainerDashboard.from\_config("dashboard.yaml")`
-

Bug Fixes

-
-

Improvements

-
-

Other Changes

-
-

Version 0.2.7:

Breaking Changes

-
-

New Features

- explainer.dump() to store explainer, explainer.from_file() to load explainer from file
- Explainer.to_yaml() and ExplainerDashboard.to_yaml() can store the configuration of your explainer/dashboard to file.
- explainerdashboard CLI:
 - Start an explainerdashboard from the command-line!
 - start default dashboard from stored explainer : `explainerdashboard run explainer.joblib`
 - start full configured dashboard from config: `explainerdashboard run explainerdashboard.yaml`
 - build explainer based on input files defined in .yaml

```
(model.pkl, data.csv, etc): `explainerdashboard build
explainerdashboard.yaml`
- includes new ascii logo :)
```

Bug Fixes

```
-
-
```

Improvements

- If idxs is not passed use X.index instead
- explainer.idxs performance enhancements
- added whatif component and tab to InlineExplainer
- added cumulative precision component to InlineExplainer

Other Changes

```
-
-
```

Version 0.2.6:

Improvements

- more straightforward imports: `from explainerdashboard import ClassifierExplainer, RegressionExplainer, ExplainerDashboard, InlineExplainer`
- all custom imports (such as ExplainerComponents, Composites, Tabs, etc) combined under `explainerdashboard.custom`:
`from explainerdashboard.custom import \*`

version 0.2.5:

Breaking Changes

```
-
-
```

New Features

- New dashboard tab: WhatIfComponent/WhatIfComposite/WhatIfTab: allows you to explore whatif scenario's by editing multiple features and observing shap contributions and pdp plots. Switch off with ExplainerDashboard parameter whatif=False.
- New login functionality: you can restrict access to your dashboard by passing a list of `[login, password]` pairs:
`ExplainerDashboard(explainer, logins=[['login1', 'password1'], ['login2', 'password2']]).run()`
- Added 'target' parameter to explainer, to make more descriptive plots.
e.g. by setting target='Fare', will show 'Predicted Fare' instead of simply 'Prediction' in various plots.
- in detailed shap/interaction summary plots, can now click on single

```

    shap value for a particular feature, and have that index highlighted
    for all features.
-   autodetecting Google colab environment and setting mode='external'
    (and suggesting so for jupyter notebook environments)
-   confusion matrix now showing both percentage and counts
-   Added classifier model performance summary component
-   Added cumulative precision component

### Bug Fixes
-
-

### Improvements
-   added documentation on how to deploy to heroku
-   Cleaned up modebars for figures
-   ClassifierExplainer asserts predict_proba attribute of model
    with model_output='logodds' still display probability in prediction
    summary
-   for ClassifierExplainer: check if has predict_proba methods at init

### Other Changes
-   removed monkeypatching shap_explainer note
-

## version 0.2.4

### New Features
-   added ExplainerDashboard parameter "responsive" (defaults to True) to
    make
        the dashboard layout responsive on mobile devices. Set it to False
    when e.g.
        running tests on headless browsers.

### Bug Fixes
-   Fixes bug that made RandomForest and xgboost explainers unpicklable

### Improvements
-   Added tests for picklability of explainers

## Version 0.2.3

### Breaking Changes
-   RandomForestClassifierExplainer and RandomForestRegressionExplainer
    will be
        deprecated: can now simply use ClassifierExplainer or
    RegressionExplainer and the
        mixin class will automatically be loaded.
-

### New Features
-   Now also support for visualizing individual trees for XGBoost models!

```

```

(XGBClassifier and XGBRegressor). The XGBExplainer mixin class will
be
    automatically loaded and make decisiontree_df(), decision_path() and
plot_trees()
    methods available, Decision Trees tab and components now also work
for
    XGBoost models.
- new parameter n_jobs for calculations that can be parallelized (e.g.
permutation importances)
- contrib_df, plot_shap_contributions: can order by global shap feature
    importance with sort='importance' (as well as 'abs', 'high-to-low'
    'low-to-high')
- added actual outcome to plot_trees (for both RandomForest and XGB)

### Bug Fixes
-
-

### Improvements
- optimized code for calculating permutation importance, adding
possibility to calculate in parallel
- shap dependence component: if no color col selected, output standard
blue dots instead of ignoring update

### Other Changes
- added selenium integration tests for dashboards (also working with
github actions)
- added tests for multiclass classification, DecisionTree and ExtraTrees
models
- added tests for XGBExplainers
- added proper docstrings to explainer_methods.py

## Version 0.2.2

### Bug Fixes
- kernel shap bug fixed
- contrib_df bug with topx fixed
- fix for shap v0.36: import approximate_interactions from shap.utils
instead of shap.common

## Version 0.2.1:
### Breaking Changes
- Removed ExplainerHeader from ExplainerComponents
    - so also removed parameter `header_mode` from ExplainerComponent
parameters
    - You can now instead synchronize pos labels across components with a
PosLabelSelector
    and PosLabelConnector.
- In regression plots instead of boolean ratio=True/False,
    you now pass residuals={'difference', 'ratio', 'log-ratio'}
- decisiontree_df_summary renamed to decisiontree_summary_df (in line
with contrib_summary_df)

```

New Features

- added check all shap values >-1 and <1 for model_output=probability
- added parameter pos_label to all components and ExplainerDashboard to set
the initial pos label
- added parameter block_selector_callbacks to ExplainerDashboard to block
the global pos label selector's callbacks. If you already have
PosLabelSelectors
in your layout, this prevents clashes.
- plot actual vs predicted now supported only logging x axis or only y
axis
- residuals plots now support option residuals='log-ratio'
- residuals-vs-col plot now shows violin plot for categorical features
- added sorting option to contributions plot/graph: sort={'abs', 'high-
to-low', 'low-to-high'}
- added final prediction to contributions plot

Bug Fixes

- Interaction connector bug fixed in detailed summary: click didn't work
- pos label was ignored in explainer.plot_pdp()
- Fixed some UX issues with interactions components

Improvements

- All `State['tabs', 'value']` condition have been taken out of
callbacks. This
used to fix some bugs with dash tabs, but seems it works even
without, so
also no need to insert dummy_tabs in `ExplainerHeader`.
- All `ExplainerComponents` now have their own pos label selector,
meaning
that they are now fully self-contained and independent. No global
dash
elements in component callbacks.
- You can define the layout of ExplainerComponents in a layout() method
instead
of _layout(). Should still define component_callbacks() to define
callbacks
so that all subcomponents that have been registered will
automatically
get their callbacks registered as well.
- Added regression `self.units` to prediction summary, shap plots,
contributions plots/table, pdp plot and trees plot.
- Clearer title for MEAN_ABS_SHAP importance and summary plots
- replace na_fill value in contributions table by "MISSING"
- add string_idx to shap and interactions summary and dependence plots,
including the violating plots
- pdp plot for classification now showing percentages instead of
fractions

Other Changes

- added hide_title parameter to all components with a title

- DecisionPathGraphComponent not available for RandomForestRegression models for now.
- In contributions graph base value now called 'population average' and colored yellow.

version 0.2:

Breaking Changes

- InlineExplainer api has been completely redefined
- JupyterExplainerDashboard, ExplainerTab and JupyterExplainerTab have been deprecated

New Features

- Major rewrite and refactor of the dashboard code, now modularized into ExplainerComponents and ExplainerComposites.
- ExplainerComponents can now be individually accessed through InlineExplainer
- All elements of components can now be switched on or off or be given an initial value.
- Makes it much, much easier to design own custom dashboards.
- ExplainerDashboard can be passed an arbitrary list of components to display as tabs.

Better docs:

- Added sections InlineExplainer, ExplainerTabs, ExplainerComponents, CustomDashboards and Deployment
- Added screenshots to documentation.

Bug Fixes

- fixes residuals y-pred instead of pred-y
-

Improvements

- Random Index Selector redesigned
- Prediction summary redesigned
- Tables now follow dbc.Table formatting
- All connections between components now happen through explicit connectors
- Layout of most components redesigned, with all elements made hideable

Other Changes

-
-

Version 0.1.13

Bug Fixes

- Fixed bug with GradientBoostingClassifier where output format of shap.expected_value was not properly accounted for.
-

```
### Improvements
- Cleaned up standalone label selector code
- Added check for shap base values to be between 0 and 1 for
model_output=='probability'
```

```
## Version 0.1.12
```

```
### Breaking Changes
- ExplainerDashboardStandaloneTab is now called ExplainerTab
-
```

```
### New Features
```

added support for the `jupyter-dash` package for inline dashboard in Jupyter notebooks, adding the following dashboard classes:

```
- `JupyterExplainerDashboard`
- `JupyterExplainerTab`
- `InlineExplainer`
```

```
## Template:
### Breaking Changes
-
-
```

```
### New Features
-
-
```

```
### Bug Fixes
-
-
```

```
### Improvements
-
-
```

```
### Other Changes
-
-
```